LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 1

ANDROID BASIC WITH COMPOSE

Oleh:

Hafiz Perdana NIM. 2410817210027

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN

LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE

MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basic With Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Hafiz Perdana
NIM : 2410817210027

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN ii
DAFTAR ISI iii
DAFTAR GAMBAR iv
DAFTAR TABEL v
SOAL 1 1
A. Source Code 3
B. Output Program 8
C. Pembahasan 9
ADDITIONAL: ESSAY 13
DAFTAR PUSTAKA 23
TAUTAN GIT 24

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi 1
Gambar 2. Tampilan Dadu Setelah Di-Roll 2
Gambar 3. Tampilan Roll Dadu Double 2
Gambar 4. Screenshot Hasil Jawaban Soal 1 XML 8
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose 9

DAFTAR TABEL

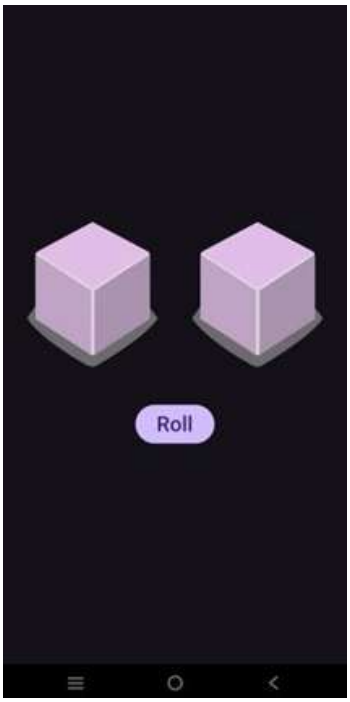
Tabel 1. Source Code MainActivity XML Soal 1 3
Tabel 2. Source Code activity_main XML Soal 1 4
Tabel 3. Source Code MainActivity Compose Soal 1 6

SOAL 1

Soal Praktikum:

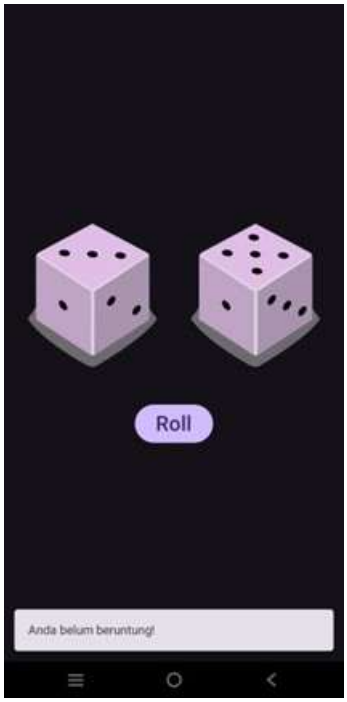
Buatlah sebuah aplikasi yang dapat menampilkan 2 (dua) buah dadu yang dapat berubah ubah tampilannya pada saat user menekan tombol "Roll". Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



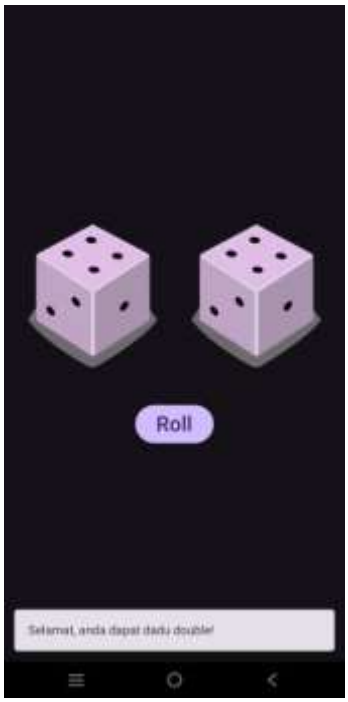
Gambar 1. Tampilan Awal Aplikasi

2. Setelah user menekan tombol "Roll" maka masing-masing dadu akan memperlihatkan sisi dadu masing-masing dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2 maka aplikasi akan menampilkan pesan "Anda belum beruntung!" seperti dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu Setelah Di-Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan "Selamat, anda dapat dadu double!" seperti yang dapat dilihat pada Gambar 3.



Gambar 3. Tampilan Roll Dadu Double

4. Buatlah aplikasi tersbeut menggunakan XML dan Jetpack Compose
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam **folder Modul 1 dalam bentuk Project.** Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut: https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp=sharing

A. Source Code

MainActivity.kt - XML

Tabel 1. Source Code MainActivity XML Soal 1

```
package com.example.modul1xml

import android.os.Bundle
import android.widget.Button
import android.widget.ImageView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val diceImage1: ImageView = findViewById(R.id.diceImage1)
        val diceImage2: ImageView = findViewById(R.id.diceImage2)
        val rollButton: Button = findViewById(R.id.rollButton)

        diceImage1.setImageResource(R.drawable.dice_0)
        diceImage2.setImageResource(R.drawable.dice_0)

        rollButton.setOnClickListener {
            val result1 = (1..6).random()
            val result2 = (1..6).random()

```

```
diceImage1.setImageResource(getDiceImage(result1))
diceImage2.setImageResource(getDiceImage(result2))

            if (result1 == result2) {
                val msgDouble = getString(R.string.double_message)
                Toast.makeText(this, msgDouble, Toast.LENGTH_SHORT).show()
            } else {
                val msgLose = getString(R.string.bad_message)
                Toast.makeText(this, msgLose, Toast.LENGTH_SHORT).show()
            }
        }
    }

    private fun getDiceImage(diceValue: Int): Int {
        return when (diceValue) {
            1 -> R.drawable.dice_1
            2 -> R.drawable.dice_2
            3 -> R.drawable.dice_3
            4 -> R.drawable.dice_4
            5 -> R.drawable.dice_5
            else -> R.drawable.dice_6
        }
    }
}
```

activity_main.xml

Tabel 2. Source Code activity_main XML Soal 1

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:background="@color/bg_white"
    tools:context=".MainActivity">

```

```
    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center">

        <ImageView
            android:id="@+id/diceImage1"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
android:contentDescription="@string/desc_dice_1"
            android:src="@drawable/dice_1" />

        <Space
            android:layout_width="16dp"
            android:layout_height="wrap_content" />

        <ImageView
            android:id="@+id/diceImage2"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
android:contentDescription="@string/desc_dice_2"
            android:src="@drawable/dice_1" />
    </LinearLayout>

    <Space
        android:layout_width="match_parent"
        android:layout_height="8dp" />

    <Button
        android:id="@+id/rollButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/roll"
        android:textAllCaps="false"
        android:backgroundTint="@color/btn_purple"
        android:textColor="@color/text_dark_purple" />

</LinearLayout>
```

MainActivity.kt - Compose

Tabel 3. Source Code MainActivity Compose Soal 1

```
package com.example.modul1compose

import android.os.Bundle
import android.widget.Toast
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.*
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.modul1compose.ui.theme.DiceRollerTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            DiceRollerTheme {
                DiceRollerApp()
            }
        }
    }
}

@Preview(showBackground = true)
@Composable
fun DiceRollerApp() {
    var leftDice by remember { mutableStateOf(0) }
    var rightDice by remember { mutableStateOf(0) }

    val context = LocalContext.current
    val msgDouble = stringResource(R.string.double_message)
    val msgLose = stringResource(R.string.bad_message)

    Column(
        modifier = Modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
```

```
        verticalArrangement = Arrangement.Center
    ) {
        Row(
            horizontalArrangement = Arrangement.Center,
            verticalAlignment = Alignment.CenterVertically
        ) {
            DiceView(leftDice)
            Spacer(modifier = Modifier.width(16.dp))
            DiceView(rightDice)
        }

        Spacer(modifier = Modifier.height(8.dp))

        Button(onClick = {
            leftDice = (1..6).random()
            rightDice = (1..6).random()

            if (leftDice == rightDice) {
                Toast.makeText(context,          msgDouble, Toast.LENGTH_SHORT).show()
            } else {
                Toast.makeText(context,            msgLose, Toast.LENGTH_SHORT).show()
            }
        }) {
            Text(stringResource(R.string.roll))
        }

    }
}

@Composable
fun DiceView(value: Int) {
    val imageRes = when (value) {
        1 -> R.drawable.dice_0
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }

    Image(
        painter = painterResource(imageRes),
        contentDescription = value.toString()
    )
```

```
}
```

B. Output Program

XML

Gambar 4. Screenshot Hasil Jawaban Soal 1 XML

Compose

Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose

C. Pembahasan

MainActivity.kt - XML

Kode ini merupakan aplikasi Android yang menampilkan dua buah gambar dadu dan sebuah tombol roll menggunakan bahasa Kotlin dengan layout berbasis XML. Program ini mensimulasikan pelemparan dua dadu secara acak dan memberikan notifikasi Toast kepada pengguna sesuai hasil yang didapat.

Pada bagian `onCreate`, komponen dari layout diambil menggunakan `findViewById`. Dua `ImageView` bernama `diceImage1` dan `diceImage2` digunakan untuk menampilkan gambar dadu, dan satu Button bernama `rollButton` digunakan sebagai tombol roll. Sebelum tombol ditekan, kedua `ImageView` sudah menampilkan gambar dadu angka satu sebagai tampilan awal saat aplikasi dibuka.

Ketika `rollButton` ditekan, `setOnClickListener` menjalankan pengacakan angka untuk tiap dadu menggunakan `(1..6).random()` yang menghasilkan angka antara 1 sampai 6. Hasilnya disimpan ke `result1` dan `result2`, lalu keduanya dikirim ke fungsi

getDiceImage untuk menentukan gambar mana yang harus ditampilkan. Gambar pada diceImage1 dan diceImage2 kemudian diperbarui sesuai angka yang keluar.

Setelah itu program mengecek apakah result1 dan result2 bernilai sama lewat percabangan if-else. Kalau sama, Toast menampilkan pesan "Selamat, anda dapat dadu double!" yang diambil dari file resource menggunakan getString, kalau tidak maka akan menampilkan pesan "Anda belum beruntung!".

Fungsi getDiceImage bertugas menerima angka hasil dadu lalu mengembalikan gambar yang sesuai menggunakan ekspresi when yang cara kerjanya mirip switch-case. Angka 1 sampai 5 masing-masing sudah punya gambar dadunya sendiri, dan angka 6 ditangani lewat blok else.

activity_main.xml

Kode ini merupakan file layout XML yang mendefinisikan tampilan antarmuka dari activity utama pada aplikasi dadu. Layout ini menggunakan LinearLayout sebagai elemen root dengan orientasi vertikal, sehingga semua elemen di dalamnya tersusun dari atas ke bawah secara berurutan. Atribut gravity diatur ke center agar seluruh konten terpusat di tengah layar baik secara horizontal maupun vertikal, dan warna latar belakang diambil dari file resource warna menggunakan referensi bg_white. Atribut tools:context pada root LinearLayout berfungsi untuk menghubungkan layout ini dengan MainActivity.

Di dalam LinearLayout utama terdapat sebuah sub LinearLayout dengan orientasi horizontal yang bertugas menampung dua gambar dadu secara berdampingan dalam satu baris. Di dalamnya terdapat dua buah ImageView dengan ID diceImage1 dan diceImage2, yang masing-masing menampilkan gambar dice_0 sebagai nilai awal saat layout pertama kali dimuat. Ukuran keduanya menggunakan wrap_content sehingga menyesuaikan ukuran gambar aslinya. Di antara kedua ImageView tersebut disisipkan elemen Space dengan lebar 16dp sebagai pemisah jarak horizontal agar kedua gambar dadu tidak berhimpitan. Atribut contentDescription pada masing-masing ImageView diisi menggunakan referensi string dari file resource untuk mendukung fitur aksesibilitas pada perangkat Android.

Setelah `LinearLayout` horizontal tersebut, terdapat elemen `Space` dengan tinggi `8dp` yang memberikan jarak vertikal antara bagian gambar dadu dengan tombol di bawahnya. Kemudian terdapat sebuah `Button` dengan ID `rollButton` yang teks labelnya diambil dari file resource string. Atribut `textAllCaps` diatur ke false agar teks tombol tidak otomatis dikonversi menjadi huruf kapital. Warna latar tombol menggunakan referensi `btn_purple` dan warna teksnya menggunakan `text_dark_purple`, keduanya didefinisikan di file resource warna.

MainActivity.kt - Compose

Kode ini merupakan implementasi aplikasi dadu yang sama seperti sebelumnya, namun dibuat menggunakan Jetpack Compose. Pada bagian `MainActivity`, fungsi `onCreate` memanggil `setContent` sebagai pengganti `setContentView` yang biasa digunakan pada pendekatan XML. Di dalam `setContent`, tema aplikasi `DiceRollerTheme` diterapkan dan fungsi `DiceRollerApp` dipanggil sebagai komponen utama yang ditampilkan ke layar.

Fungsi `DiceRollerApp` adalah composable utama yang memuat tampilan sekaligus logika aplikasi. Di sini dideklarasikan dua variabel `state` yaitu `leftDice` dan `rightDice` menggunakan `mutableStateOf` yang dibungkus dengan `remember`, sehingga nilainya tetap diingat dan tampilan akan otomatis diperbarui setiap kali nilainya berubah. `Context` diambil menggunakan `LocalContext.current` untuk keperluan Toast, dan teks pesan diambil dari file resource menggunakan `stringResource`.

Susunan tampilannya menggunakan Column yang memenuhi seluruh layar dengan konten terpusat di tengah. Di dalamnya terdapat Row yang menampung dua komponen `DiceView` secara berdampingan dengan Spacer selebar `16dp` di antaranya. Di bawah Row terdapat Spacer setinggi `8dp`, lalu sebuah Button yang saat ditekan mengacak angka untuk `leftDice` dan `rightDice` menggunakan `(1..6).random()`, kemudian mengecek apakah keduanya bernilai sama untuk menentukan pesan Toast yang muncul.

Fungsi `DiceView` adalah composable terpisah yang menampilkan gambar dadu sesuai nilai yang diterima lewat parameter value. Ekspresi when di dalamnya menentukan resource gambar yang sesuai dengan angkanya, termasuk nilai 0 yang dipetakan ke `dice_0` sebagai

tampilan awal sebelum tombol roll pertama kali ditekan. Gambar ditampilkan menggunakan komponen Image dengan `painterResource` untuk memuat gambar dari drawable.

ADDITIONAL:

ESSAY

1. Riset dan *cite* **artikel/jurnal/buku** mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatkan opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan. **
2. Apa yang membedakan OOP pada kotlin dengan OOP pada java? *
3. Sebutkan, dan jelaskan mengenai SOLID Principles *
4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

**** = 2 sumber**
*** = 1 sumber**
Gunakanlah sumber primary artikel/jurnal/buku, web hanya sebagai penunjang opini/argumen

---

1. Imperative vs. Declarative Programming dalam Pembuatan UI

Pengertian Dasar

Sebelum masuk ke konteks UI, penting untuk memahami apa bedanya dua paradigma ini secara konseptual. Menurut Samuel [1], dalam pembuatan software terdapat berbagai paradigma yang terus berkembang, dan di antara yang paling dominan adalah imperative programming, di mana programmer menginstruksikan komputer apa yang diinginkan sekaligus bagaimana cara mendapatkannya, langkah demi langkah secara eksplisit. Sebaliknya, declarative programming fokus ke apa yang ingin dicapai, tanpa harus ngurusin detail implementasinya.

Kalau mau pakai analogi: imperative itu kayak ngasih resep masak yang detail banget langkah demi langkahnya, sedangkan declarative itu kayak pesan di restoran, kamu bilang mau makan apa, dan koki yang ngurusin prosesnya.

Penerapannya di Pembuatan UI

Dalam konteks UI development, perbedaan ini jadi sangat terasa. Samuel [1] secara khusus membahas bahwa Event-Driven with GUI programming adalah salah satu paradigma mainstream yang dominan di industri, dan paradigma ini pada dasarnya bersifat imperative,

di mana developer harus mendefinisikan isi, tema, posisi komponen, dan mengelola event handler secara eksplisit. Lebih lanjut, Samuel [1] menjelaskan bahwa elemen-elemen GUI pada berbagai platform (Android, iOS, Windows) memiliki banyak kesamaan, namun cara pengaturannya bergantung sepenuhnya pada keputusan developer secara manual, sebuah karakteristik khas pendekatan imperative.

Contoh konkretnya bisa dilihat di Android development. Dengan cara lama (imperative, pakai Android Views), kalau mau nampilin teks "Hello, Omar!", kita harus findViewById, lalu set .text-nya secara manual. Kalau ada perubahan data, kita harus nyari lagi komponen yang relevan dan update sendiri.

Di sisi lain, framework declarative modern seperti Jetpack Compose atau SwiftUI mengubah cara kerja ini secara fundamental. Seperti yang dipaparkan Hanus dan Kluss [2], pendekatan ini mengusulkan deskripsi deklaratif untuk user interface yang mengabstraksi detail implementasi level rendah, di mana framework yang mereka kembangkan bahkan dapat dijalankan sebagai GUI desktop maupun web user interface melalui browser standar. Ini membuktikan bahwa ide declarative UI sebenarnya sudah lama dibicarakan di komunitas ilmiah jauh sebelum era Jetpack Compose dan SwiftUI populer.

Kelebihan dan Kekurangan Masing-masing Paradigma

Imperative punya kelebihan tersendiri: developer punya kontrol penuh atas setiap aspek UI. Samuel [1] mencatat bahwa dalam paradigma imperative, developer dapat dengan bebas memilih dan mengombinasikan berbagai konsep GUI sesuai kebutuhan, dan ini berguna kalau kita perlu optimasi performa yang spesifik atau implementasi behavior yang sangat custom. Tapi downside-nya juga nyata karena proyek imperative cenderung lebih sulit di-maintain akibat banyaknya mutable state, prosedur yang tidak jelas, dan dependensi yang implisit.

Sementara declarative punya nilai jual yang cukup kuat. Menurut Hanus dan Kluss [2], framework declarative dapat mengelola rendering lintas platform (desktop dan web) dari satu deskripsi yang sama, sehingga developer cukup mendeskripsikan tampilan UI berdasarkan state yang diinginkan dan framework yang mengurus cara mewujudkannya. Pendekatan ini juga mengeliminasi proses sinkronisasi manual antara UI dan state aplikasi.

Opini: Paradigma Mana yang Lebih Saya Prefer?

Kalau ditanya mana yang lebih oke, menurut saya declarative programming lebih menarik, terutama untuk konteks pengembangan UI modern. Alasannya sederhana: kompleksitas UI zaman sekarang jauh lebih tinggi dibanding 10 sampai 15 tahun lalu. State management di app modern bisa sangat complex, ada user interaction, data dari network, animasi, dan lain-lain. Kalau harus ngurusin semua itu secara manual satu per satu kayak di imperative, rasanya akan sangat melelahkan dan error-prone.

Samuel [1] sendiri mengingatkan bahwa desain software modern sudah tidak bisa diselesaikan dengan satu paradigma saja, dan di sinilah declarative tampil sebagai pelengkap yang kuat. Pengalaman ngoding dengan Jetpack Compose (yang declarative) rasanya jauh lebih intuitif dibanding XML + View binding yang imperative: kita fokus ke "UI ini harusnya tampil kayak apa kalau state-nya begini", bukan "gimana caranya ubah komponen ini kalau ada perubahan data".

Tapi tentu bukan berarti imperative tidak relevan. Untuk skenario tertentu yang butuh kontrol sangat granular, atau ketika bekerja dengan codebase legacy yang besar, imperative masih punya tempatnya. Di industri pun, banyak framework modern menyediakan fallback ke primitif imperative ketika API declarative belum tersedia.

Referensi

[1] M. S. Samuel, "An Insight into Programming Paradigms and Their Programming Languages," *Journal of Applied Technology and Innovation,* vol. 1, no. 1, pp. 37-57, 2017. Dapat diakses di: https://dif7uuh3zqcps.cloudfront.net/wp-content/uploads/sites/11/2018/07/17035708/2017_Issue1_Paper4.pdf

[2] M. Hanus and C. Kluß, "Declarative Programming of User Interfaces," in *Practical Aspects of Declarative Languages*, Berlin, 2009. Dapat diakses di: https://www.researchgate.net/publication/220802929_Declarative_Programming_of_User_Interfaces

2. Perbedaan OOP pada Kotlin dengan OOP pada Java

Kotlin dan Java sama-sama berbasis JVM dan mendukung Object-Oriented Programming. Keduanya punya konsep class, object, inheritance, encapsulation, dan polymorphism. Tapi terlepas dari kesamaan fondasi itu, ada beberapa perbedaan cukup signifikan dalam cara OOP diimplementasikan di kedua bahasa ini.

1) Null Safety sebagai Bagian dari Type System

Salah satu perbedaan paling mencolok adalah cara keduanya handle null. Di Java, null bisa masuk ke mana saja tanpa peringatan dari compiler, dan hasilnya adalah NullPointerException yang terkenal "ramah" muncul di runtime. Menurut Daniela Gotseva, dkk [3], null safety adalah fitur Kotlin yang dirancang untuk secara signifikan mengurangi risiko null reference, di mana tipe data non-nullable dan nullable dibedakan secara eksplisit dalam type system, sehingga potensi NPE bisa ditangkap di waktu kompilasi, bukan di runtime.

Konkretnya di Kotlin, kalau kita tulis val name: String, itu dijamin tidak bisa null. Kalau memang mau nullable, harus eksplisit tulis val name: String?. Bandingkan dengan Java di mana semua reference type bisa null secara default.

2) Sintaks OOP yang Lebih Ringkas

Di Java, membuat class dengan properti dan constructor membutuhkan banyak boilerplate: getter, setter, dan constructor yang dideklarasikan terpisah. Daniela Gotseva, dkk [3] secara eksplisit menyatakan bahwa Kotlin unggul dalam hal keringkasan serta fungsionalitas kode dibandingkan Java. Constructor sudah terintegrasi dalam deklarasi class sebagai primary constructor, sehingga class yang di Java butuh puluhan baris cukup ditulis dalam satu baris di Kotlin: class Person(val name: String, val age: Int).

3) Extension Functions

Ini adalah fitur yang tidak ada di Java tapi ada di Kotlin, dan sangat relevan dalam OOP. Daniela Gotseva, dkk [3] mencatat bahwa extension functions memungkinkan developer menambahkan fungsi baru ke class yang sudah ada tanpa harus memodifikasi source code class tersebut. Ini berguna sekali karena kita bisa "memperluas" class dari library pihak ketiga tanpa harus inheritance atau wrapper. Di

Java, kalau mau menambahkan behavior ke String, harus buat utility class terpisah, sementara di Kotlin bisa langsung dideklarasikan sebagai extension function.

4) Companion Object vs Static

Di Java, kita pakai keyword static untuk member yang terkait dengan class, bukan instance. Kotlin tidak punya static. Sebagai gantinya, Kotlin menggunakan companion object, yaitu sebuah object yang dideklarasikan di dalam class dan bisa mengakses private member class tersebut. Ini sebetulnya lebih OOP karena companion object sendiri adalah instance dari suatu tipe.

5) Data Class

Java tidak punya padanan langsung dari data class Kotlin, setidaknya sebelum Java 16 dengan Records. Ini membuat penggunaan class sebagai wadah data jauh lebih efisien di Kotlin, yang akan dibahas lebih lanjut di bagian 4.

Secara keseluruhan, sebagaimana disimpulkan oleh Daniela Gotseva, dkk [3], Kotlin adalah salah satu dari sekian bahasa yang menggunakan JVM, namun dalam waktu singkat berhasil mendapatkan popularitas yang signifikan dibandingkan Java yang sudah mendominasi selama beberapa dekade, salah satunya karena penyempurnaan fitur OOP yang lebih modern dan aman.

Referensi

[3] D. Gotseva, Y. Tomov and P. Danov, "Comparative Study Java vs Kotlin," Sofia, 2019. Dapat diakses di: https://ieeexplore.ieee.org/document/8994896

3. SOLID Principles

SOLID adalah kumpulan lima prinsip desain software yang jadi semacam "panduan hidup" buat developer yang mau nulis kode yang bersih, mudah di-maintain, dan fleksibel. Ramachandrappa [4] menjelaskan bahwa prinsip-prinsip ini pertama kali diperkenalkan oleh Robert C. Martin dalam paper-nya tahun 2000 untuk membantu developer menulis software yang mudah dipahami, dimodifikasi, dan dikembangkan, dengan akronim SOLID kemudian dipopulerkan oleh Michael Feathers.

S - Single Responsibility Principle (SRP)

Menurut Ramachandrappa [4], sebuah class harus punya satu dan hanya satu alasan untuk berubah, artinya sebuah class seharusnya hanya punya satu tanggung jawab saja. Ini mungkin prinsip yang paling sering dilanggar tanpa disadari. Misalnya class UserManager yang sekaligus handle autentikasi, simpan data ke database, dan kirim email notifikasi, itu jelas terlalu banyak tanggung jawab. Solusinya adalah memisahkan ke class yang berbeda sesuai tanggung jawabnya: AuthService untuk autentikasi, UserRepository untuk akses data, dan NotificationService untuk kirim notifikasi.

O - Open/Closed Principle (OCP)

Ramachandrappa [4] mendefinisikan prinsip ini sebagai: entitas software (class, module, function, dll.) harus terbuka untuk ekstensi, tetapi tertutup untuk modifikasi. Artinya, ketika ada requirement baru, idealnya kita bisa tambahkan fitur tanpa harus mengubah kode yang sudah ada dan sudah teruji. Ini biasanya dicapai dengan abstraksi, yaitu interface atau abstract class yang bisa diimplementasikan oleh berbagai class konkret. Dengan mengizinkan entitas software untuk diperluas tanpa modifikasi, developer bisa menambahkan fungsionalitas baru tanpa risiko merusak kode yang sudah ada.

L - Liskov Substitution Principle (LSP)

Masih menurut Ramachandrappa [4], object dalam program harus bisa digantikan oleh instance dari subtype-nya tanpa mengubah kebenaran program tersebut. Sederhananya, kalau class B adalah subclass dari A, maka di mana pun kita pakai objek A, kita harus bisa pakai objek B tanpa ada yang "rusak". Ini bukan sekadar soal inheritance secara teknis, tapi soal

apakah subclass benar-benar “bersikap” seperti parent class-nya. Contoh klasik yang melanggar LSP: Penguin adalah subclass dari Burung, tapi kalau Burung punya method terbang(), maka Penguin tidak bisa terbang dan substitusi akan menghasilkan behavior yang tidak terduga.

I - Interface Segregation Principle (ISP)

Ramachandrappa [4] juga menjelaskan bahwa client tidak boleh dipaksa bergantung pada interface yang tidak mereka gunakan. Prinsip ini menghindari pembuatan “fat interface”, yaitu interface yang terlalu besar dengan banyak method yang tidak selalu relevan untuk semua implementornya. Lebih baik membuat beberapa interface kecil yang lebih spesifik daripada satu interface besar. Contohnya, daripada interface Worker yang punya method work(), eat(), dan sleep(), yang tidak masuk akal untuk robot, lebih baik pisahkan menjadi Workable, Eatable, dan Sleepable.

D - Dependency Inversion Principle (DIP)

Terakhir, Ramachandrappa [4] menegaskan bahwa high-level module tidak boleh bergantung pada low-level module, keduanya seharusnya bergantung pada abstraksi. Abstraksi juga tidak boleh bergantung pada detail, melainkan detail yang harus bergantung pada abstraksi. Intinya adalah jangan hardcode dependensi ke implementasi konkret, melainkan gunakan abstraksi (interface) sehingga implementasinya bisa diganti tanpa mengubah high-level logic. Dalam praktiknya, ini sering diwujudkan lewat dependency injection: daripada class OrderService langsung membuat instance MySQLDatabase, lebih baik OrderService menerima interface Database dari luar, sehingga kalau mau ganti ke PostgreSQL, OrderService-nya tidak perlu diubah sama sekali.

Secara keseluruhan, sebagaimana ditegaskan oleh Ramachandrappa [4], menerapkan prinsip-prinsip SOLID dalam pengembangan aplikasi menghasilkan kode yang lebih bersih, lebih mudah di-maintain, dan lebih skalabel.

Referensi

[4] N. C. Ramachandrappa, "SOLID Design Principles in Software Engineering," *International Journal of Computer Trends and Technology,* vol. 72, no. 9, pp. 18-23, 2024. Dapat diakses di: https://ijcttjournal.org/Volume-72%20Issue-9/IJCTT-V72I9P104.pdf

4. Perbedaan Class dan Data Class pada Kotlin

Di Kotlin, baik class maupun data class sama-sama bisa dipakai untuk mendefinisikan blueprint suatu object. Tapi ada perbedaan fundamental yang membuat keduanya punya use case yang berbeda.

Class Biasa

Class biasa di Kotlin dipakai ketika kita perlu mendefinisikan object yang punya perilaku kompleks, logic bisnis, atau state yang berubah-ubah. Dokumentasi resmi Kotlin [5] menjelaskan bahwa sebuah class biasa bisa memiliki constructor tanpa parameter dan bisa di-extend oleh class lain. Kalau kita buat dua instance class biasa dengan properti yang sama persis, perbandingan dengan == akan mengembalikan false secara default, karena operator == di class biasa membandingkan referensi memori (reference equality), bukan isi nilainya.

Data Class

Berbeda dengan class biasa, dokumentasi resmi Kotlin [5] menjelaskan bahwa data class dirancang khusus untuk menyimpan data. Untuk setiap data class, compiler secara otomatis men-generate beberapa member function tambahan yang memungkinkan kita untuk mencetak instance ke output yang readable, membandingkan instance, menyalin instance, dan lain-lain. Method yang di-generate otomatis meliputi equals() untuk membandingkan berdasarkan nilai properti bukan referensi, hashCode() yang konsisten dengan equals(), toString() yang menghasilkan output readable seperti Person(name=Alice, age=30), copy() untuk membuat salinan object dengan kemungkinan mengubah beberapa properti, serta componentN() untuk keperluan destructuring declaration.

Perbedaan ini sangat terlihat dalam perbandingan objek: pada class biasa, dua objek dengan nilai yang sama tidak akan dianggap setara, sementara pada data class perbandingan menghasilkan true karena equality berbasis nilai.

Ketentuan Data Class

Dokumentasi resmi Kotlin [5] menetapkan beberapa syarat yang harus dipenuhi: data class harus dideklarasikan dengan minimal satu parameter di primary constructor yang dideklarasikan dengan val atau var. Data class juga tidak bisa di-extend oleh class lain karena bersifat final secara default, dan tidak bisa bersifat sealed, open, abstract, atau inner.

Fungsi copy() yang Powerful

Salah satu fitur paling berguna dari data class adalah fungsi copy(). Masih menurut dokumentasi resmi Kotlin [5], fungsi copy() sangat berguna saat bekerja dengan objek yang immutable, karena kita bisa membuat versi baru dari objek tersebut dengan beberapa properti yang diubah, sementara objek aslinya tetap tidak termodifikasi. Contohnya, kalau kita punya val user = User(1, “Alice”, 28) dan mau buat versi baru dengan nama berbeda, cukup user.copy(name = “Bob”) tanpa perlu membuat instance baru dari nol.

Referensi

[5] Kotlin Foundation, "Data Classes," JetBrains, 2024. [Online].
Dapat diakses di: https://kotlinlang.org/docs/data-classes.html

DAFTAR PUSTAKA

[1] M. S. Samuel, "An Insight into Programming Paradigms and Their Programming Languages," *Journal of Applied Technology and Innovation,* vol. 1, no. 1, pp. 37-57, 2017.

[2] M. Hanus and C. Kluß, "Declarative Programming of User Interfaces," in *Practical Aspects of Declarative Languages*, Berlin, 2009.

[3] D. Gotseva, Y. Tomov and P. Danov, "Comparative Study Java vs Kotlin," Sofia, 2019.

[4] N. C. Ramachandrappa, "SOLID Design Principles in Software Engineering," *International Journal of Computer Trends and Technology,* vol. 72, no. 9, pp. 18-23, 2024.

[5] Kotlin Foundation, "Data Classes," JetBrains, 2024. [Online]. Available: https://kotlinlang.org/docs/data-classes.html. [Accessed April 2025].

TAUTAN GIT

Berikut adalah tautan untuk repositori GitHub yang telah dibuat

https://github.com/Fiezz65/MODUL-MOBILE

Berikut adalah tautan untuk source code yang telah dibuat.

MainActivity.kt - XML:

https://github.com/Fiezz65/MODUL-MOBILE/blob/master/MODUL_1/XML/app/src/main/java/com/example/modul1xml/MainActivity.kt

activity_main.xml:

https://github.com/Fiezz65/MODUL-MOBILE/blob/master/MODUL_1/XML/app/src/main/res/layout/activity_main.xml

MainActivity.kt - Compose:

https://github.com/Fiezz65/MODUL-MOBILE/blob/master/MODUL_1/COMPOSE/app/src/main/java/com/example/modul1compose/MainActivity.kt